CHƯƠNG 7

Thiết kế hệ thống và dữ liệu cho ứng dụng AI tạo sinh

[Ứng dụng Trí tuệ nhân tạo]



[Nguyễn Tuấn Anh]
• [Thái Nguyên, 2026]

Giới thiệu nội dung

Mục tiêu chương

- Mô tả được sự khác biệt căn bản về tổ chức dữ liệu giữa ứng dụng truyền thống và ứng dụng AI tạo sinh, cùng bảy nguyên tắc thiết kế kiến trúc dữ liệu cho GenAI.
- Phân tích được mô hình ba tầng dữ liệu, giải thích được nguyên lý hoạt động của RAG, và so sánh được Traditional RAG với Agentic RAG.
- Thiết kế được pipeline dữ

Nội dung chính

1. Tổ chức và quản lý dữ liệu
2. Mô hình RAG và cơ sở dữ liệu vector
3. Thiết kế pipeline dữ liệu
4. Triển khai, tối ưu và thực hành

Khởi động: Dữ liệu trong ứng dụng AI có gì khác?

Ứng dụng truyền thống – Quản lý bán hàng:

- Bảng `khach_hang`, `don_hang`, `san_pham` – **cột rõ ràng**, quan hệ khóa ngoại, ràng buộc toàn vẹn.
- Truy vấn chính xác bằng SQL.
- Logic nghiệp vụ **nhúng trong code**.

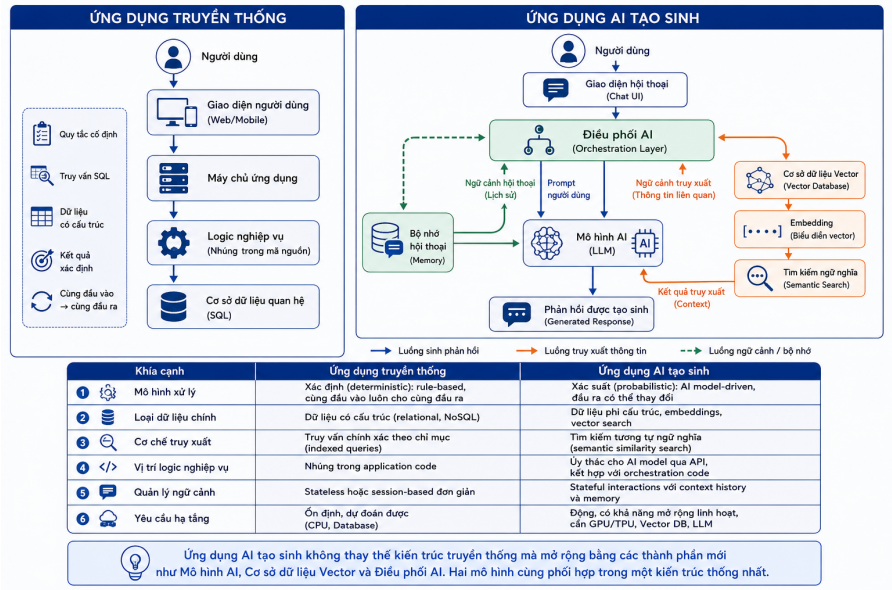
Ứng dụng AI tạo sinh – Hỏi đáp chính sách:

- Dữ liệu nằm trong **PDF, email, slide, web** – phi cấu trúc, không chuẩn hóa.
- **Không thể** truy vấn bằng SQL.
- Logic xử lý được **ủy thác cho AI model** qua API.

Câu hỏi then chốt

Làm thế nào để tổ chức dữ liệu phi cấu trúc thành "nhiên liệu tri thức" mà AI có thể hiểu và truy xuất theo thời gian thực?

Khác biệt căn bản: Truyền thống và GenAI



Khía cạnh	Ứng dụng truyền thống	Ứng dụng AI tạo sinh
1 Mô hình xử lý	Xác định (deterministic): rule-based, cùng đầu vào luôn cho cùng đầu ra	Xác suất (probabilistic): AI model-driven, đầu ra có thể thay đổi
2 Loại dữ liệu chính	Dữ liệu có cấu trúc (relational, NoSQL)	Dữ liệu phi cấu trúc, embeddings, vector search
3 Cơ chế truy xuất	Truy vấn chính xác theo chỉ mục (indexed queries)	Tìm kiếm tương tự ngữ nghĩa (semantic similarity search)
4 Vị trí logic nghiệp vụ	Nhúng trong application code	Ủy thác cho AI model qua API, kết hợp với orchestration code
5 Quản lý ngữ cảnh	Stateless hoặc session-based đơn giản	Stateful interactions với context history và memory
6 Yêu cầu hạ tầng	Ổn định, dự đoán được (CPU, Database)	Động, có khả năng mở rộng linh hoạt, cần GPU/TPU, Vector DB, LLM

Ứng dụng AI tạo sinh không thay thế kiến trúc truyền thống mà mở rộng bằng các thành phần mới như Mô hình AI, Cơ sở dữ liệu Vector và Điều phối AI. Hai mô hình cùng phối hợp trong một kiến trúc thống nhất.

Kiến trúc truyền thống (trái) và GenAI (phải):

AI Model, Vector DB và Orchestration layer là những thành phần mới.

- **Mô hình xử lý**: Deterministic (xác định) → Probabilistic (xác suất).
- **Loại dữ liệu**: Có cấu trúc → Phi cấu trúc + embeddings.
- **Truy xuất**: Chỉ mục chính xác → Tìm kiếm tương tự ngữ nghĩa.
- **Logic**: Nhúng trong code → Ủy thác cho AI model qua API.
- **Hạ tầng**: Ổn định, dự đoán được → Động, cần GPU/TPU.

Bảy nguyên tắc thiết kế kiến trúc dữ liệu GenAI (1/2)

1. **Module hóa và hướng API**: Mỗi thành phần (retrieval, processing, inference, storage) là module độc lập, giao tiếp qua API rõ ràng.
   - ⊕ Phát triển, kiểm thử, scale độc lập; cô lập phụ thuộc công nghệ; dễ A/B test.
2. **Xử lý dữ liệu có ngữ cảnh**: Hệ thống phải hiểu và tận dụng context – xác định ngữ cảnh liên quan, kết hợp đa nguồn, quản lý giới hạn context window.
   - Cơ chế chính: **RAG** – chỉ truy xuất đoạn thực sự liên quan thay vì nhồi nhét toàn bộ tài liệu.
3. **Suy luận mô hình scalable và hiệu quả**: Inference là điểm tốn tài nguyên nhất – cần batch processing, caching, quantization, GPU acceleration.
4. **Học liên tục và thích ứng mô hình**: Cơ chế để mô hình liên tục được cập nhật – thu thập phản hồi, đánh giá định kỳ, MLOps, cập nhật cơ sở tri thức không cần deploy lại.

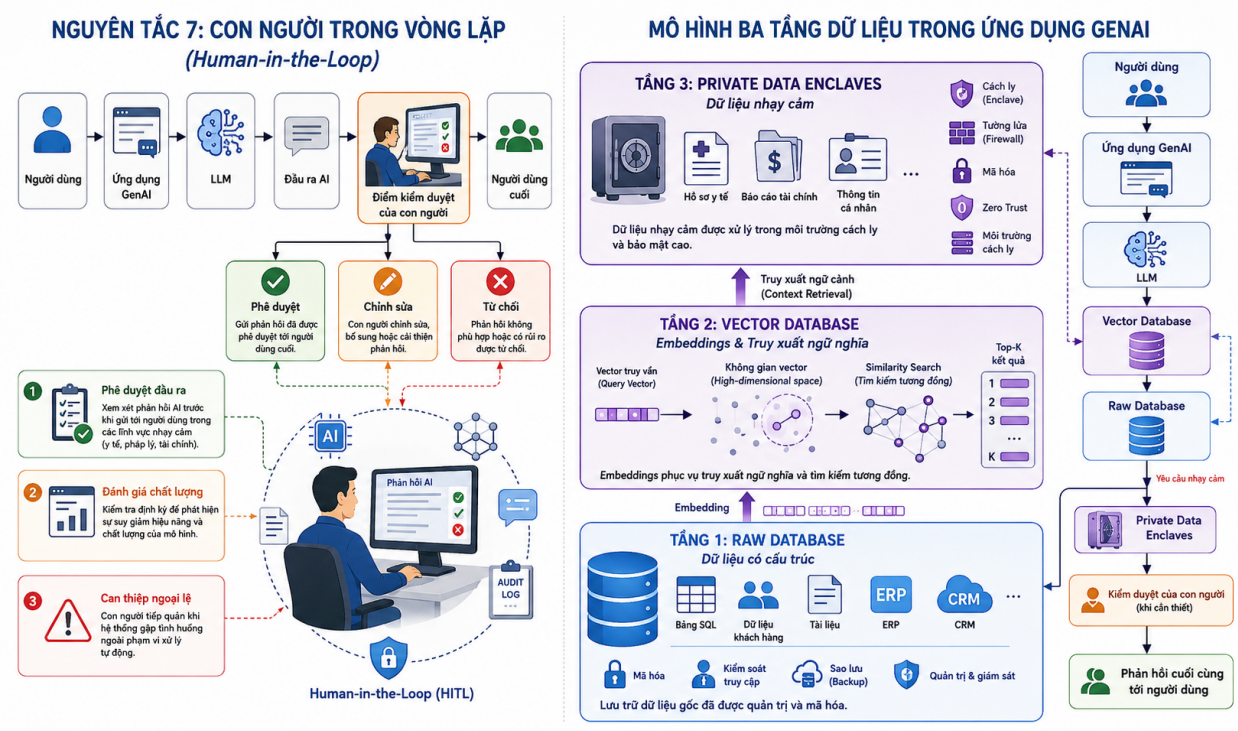
Bảy nguyên tắc thiết kế kiến trúc dữ liệu GenAI (2/2)

5. **Bảo mật, tuân thủ và AI có đạo đức**: Mã hóa in transit + at rest; kiểm soát truy cập (least privilege); lọc nội dung; tuân thủ GDPR, quy định Việt Nam; audit logging.
6. **Triển khai linh hoạt, tương thích đa nền tảng**: Vận hành trên cloud, on-premise, managed services hoặc self-hosted – tùy yêu cầu về chủ quyền dữ liệu, chi phí, tránh vendor lock-in.
7. **Con người trong vòng lặp (HITL – Human-in-the-Loop)**:
   - Thiết kế các điểm kiểm soát nơi con người can thiệp vào quyết định của AI.
   - Xem xét/phê duyệt đầu ra (y tế, pháp lý); đánh giá chất lượng định kỳ; can thiệp khi hệ thống gặp tình huống ngoài khả năng.

Lưu ý

Bảy nguyên tắc là **khung tư duy**, không phải danh sách kiểm tra cứng nhắc. Cân nhắc theo từng bối cảnh cụ thể.

Mô hình ba tầng dữ liệu trong ứng dụng GenAI



Tầng 1: Raw Database

Dữ liệu người dùng & nghiệp vụ. PostgreSQL, MongoDB. Mã hóa, backup, kiểm soát

Tầng 2: Vector Database

Embeddings cho similarity search. Pinecone, Milvus, Chroma. Dữ liệu ở dạng

Tầng 3: Private Enclaves

Dữ liệu cực nhạy cảm – môi trường cách ly, on-premise, không gọi API ngoài.

Tổ chức kho tri thức và chiến lược chunking

Kho tri thức (Knowledge Store):

- **Vector DB**: embedding + similarity search.
- **SQL DB**: metadata có cấu trúc – hỗ trợ lọc.
- **NoSQL**: nội dung thô phi cấu trúc.
- **Object Storage**: file thô (S3, GCS).

Ba chiến lược chunking:

1. **Semantic**: theo ranh giới ý nghĩa – chất lượng tốt nhất.
2. **Recursive**: theo cấu trúc phân cấp – bảo toàn logic.
3. **Paragraph**: theo đoạn văn – đơn giản, dễ triển khai.

Không có chiến lược "tốt nhất" – phụ thuộc loại tài liệu, câu hỏi, embedding model. **Luôn thử nghiệm** với dữ liệu thực tế.

Hạ tầng cho ứng dụng AI tạo sinh

Ba miền hạ tầng cốt lõi:

Tài nguyên tính toán

- GPU/TPU cho inference & training.
- On-demand scaling.
- High-performance computing cho embedding generation hàng loạt.

Tích hợp hệ thống

- Pipeline dữ liệu độ trễ thấp.
- Workload orchestration.
- API gateway – quản lý và bảo vệ endpoint.

Quản trị (Governance)

- Monitoring & alerting toàn hệ thống.
- Theo dõi chi phí token.
- Quản lý phiên bản mô hình & dữ liệu.
- Tuân thủ chính sách bảo mật.

Tổng kết Section 1

Dữ liệu trong GenAI khác biệt căn bản: phi cấu trúc, xác suất, ngữ

Khởi động: Khi mô hình ngôn ngữ không đủ

Thí nghiệm tư duy

Bạn hỏi ChatGPT: *"Chính sách nghỉ phép năm 2026 của công ty tôi là gì?"*

- Nếu may mắn: mô hình thừa nhận không biết.
- Nếu không: mô hình **bịa ra** một chính sách nghe hợp lý nhưng hoàn toàn sai – **hallucination**.

Vấn đề cốt lõi: LLM chỉ có kiến thức "đóng băng" tại thời điểm huấn luyện.

- Không biết thông tin thay đổi sau huấn luyện.
- Không truy cập được dữ liệu nội bộ tổ chức.
- Không phân biệt được "tôi biết" và "tôi đang bịa".

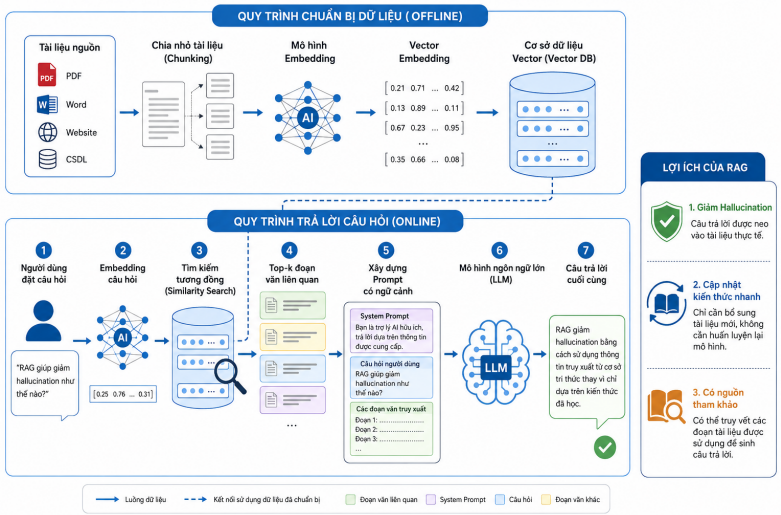
RAG = giải pháp: Cho phép mô hình "tra cứu" thông tin từ nguồn bên

Nguyên lý hoạt động của RAG

RAG = Retrieval (truy xuất) + Generation (sinh nội dung)

1. **Người dùng đặt câu hỏi** → embedding model tạo vector truy vấn.
2. **Truy xuất**: tìm top-k chunks gần nhất trong vector database (similarity search).
3. **Xây dựng prompt**: kết hợp system prompt + chunks truy xuất + câu hỏi gốc.
4. **LLM sinh câu trả lời**: đã được "tiếp đất" (grounded) bởi thông tin thực tế.

Ba lợi ích chính:



Trên: quy trình offline (index hóa tài liệu). Dưới: quy trình online (hỏi đáp thời gian thực).

Ba mức độ phát triển của ứng dụng hỏi đáp

Khía cạnh	Mức 1: Simple Prompt	Mức 2: Context-Augmented	Mức 3: RAG
Nguồn kiến thức	Chỉ từ huấn luyện LLM	Huấn luyện + ngữ cảnh tĩnh	Huấn luyện + **động**
Cập nhật	Cần fine-tune hoặc train lại	Sửa ngữ cảnh thủ công	Cập nhật cơ s đủ
Độ chính xác (dữ liệu nội bộ)	Rất thấp – dễ hallucina-tion	Trung bình – chỉ tốt câu phổ biến	**Cao** – luôn c phù hợp
Sử dụng context window	Tối thiểu	Lãng phí cho ngữ cảnh không liên quan	**Tối ưu** – ch thông tin liên c
Trích dẫn nguồn	Không	Hạn chế	Có thể truy x gốc

Mức 3 (RAG) vượt trội cả ba tiêu chí: độ chính xác, khả năng cập nhật và trích dẫn nguồn. Là lựa chọn cho ứng dụng doanh nghiệp.

Traditional RAG và Agentic RAG

Traditional RAG

- **Luồng cố định**: câu hỏi → embedding → truy xuất → prompt → LLM → kết quả.
- Tốt cho câu hỏi đơn giản, câu trả lời có trực tiếp trong tài liệu.
- Hạn chế: không xử lý được câu hỏi phức tạp, suy luận nhiều bước.

Agentic RAG

- **AI agent ra quyết định động**: đánh giá chất lượng truy xuất, chọn công cụ phù hợp.
- Tích hợp **đa công cụ**: vector search, web search, SQL query, API call, code execution.
- Thích ứng thời gian thực: truy vấn lại nếu cần; tự học từ tương tác trước.

Câu hỏi thiết kế

Agentic RAG có thực sự cần thiết cho mọi ứng dụng? Hay Traditional RAG với quy trình kiểm soát chặt đã là đủ?

Kiến trúc sáu bước của Agentic RAG

1. **Xử lý câu hỏi người dùng**: Phân tích độ phức tạp, xác định loại thông tin cần, có thể hỏi ngược lại để làm rõ.
2. **Truy xuất dữ liệu động**: Chọn nguồn phù hợp – vector DB (ngữ nghĩa), SQL DB (có cấu trúc), web search (thời gian thực), API (chuyên biệt). Có thể truy vấn song song nhiều nguồn.
3. **Xử lý tầng tri thức**: Lọc nhiễu, loại trùng lặp, reranking, fusion – tổng hợp thành biểu diễn tri thức có cấu trúc.
4. **Ra quyết định tự động**: Đánh giá thông tin đã đủ chưa? Nếu chưa – truy vấn lại, yêu cầu thêm input, hoặc chia nhỏ câu hỏi.
5. **Thực thi hành động**: Sinh câu trả lời, gọi API thực hiện tác vụ, hoặc tạo báo cáo tổng hợp.
6. **Ghi nhớ và học hỏi**: Lưu kinh nghiệm – chiến lược nào hiệu quả? Thông tin nào hữu ích? Cải thiện theo thời gian.

Đây không phải pipeline tuyến tính cứng nhắc – agent có thể quay lại, lặp lại hoặc bỏ qua bước tùy tình huống.

Vector database – “Trái tim” của RAG

- **Nguyên lý**: Lưu trữ vector embedding và tìm kiếm theo **độ tương tự** (similarity), không phải khớp chính xác (exact match).
- **Hai độ đo phổ biến**:
  - **Cosine Similarity**: đo góc giữa hai vector – phổ biến nhất cho tìm kiếm văn bản.
  - **Euclidean Distance**: đo khoảng cách hình học – phù hợp khi độ lớn vector mang ý nghĩa.

Các giải pháp phổ biến:

- **Pinecone**: Managed cloud, scale tự động – startup, dự án vừa/nhỏ.
- **Milvus**: Self-hosted, hiệu năng cao, tỷ vector – hệ thống quy mô lớn.
- **Weaviate**: Managed/Self-hosted, hybrid search (vector + keyword).
- **Chroma**: Embedded, nhẹ, đơn giản – prototype, phát triển.
- **pgvector**: PostgreSQL extension – tận dụng DB có sẵn.
- **Qdrant**: Self-hosted/Cloud, HNSW, lọc metadata mạnh.

Tiêu chí lựa chọn vector database

Sáu tiêu chí cần cân nhắc:

1. **Quy mô dữ liệu**: Hàng nghìn hay hàng tỷ vector? Milvus cho quy mô lớn; Chroma/pgvector cho quy mô nhỏ.
2. **Yêu cầu độ trễ**: Managed service ổn định hơn; self-hosted có thể tối ưu cho phần cứng cụ thể.
3. **Ngân sách và chi phí vận hành**: Managed tính theo usage; self-hosted cần chi phí hạ tầng + nhân lực.
4. **Kỹ năng đội ngũ**: Không có kinh nghiệm vận hành infrastructure? Chọn managed service.
5. **Yêu cầu tích hợp**: Cần hybrid search (vector + keyword)? Cần lọc metadata phức tạp?
6. **Bảo mật và compliance**: Dữ liệu phải nằm trong hạ tầng của bạn? Bắt buộc self-hosted.

Tổng kết Section 2

RAG là cầu nối giữa tổ chức dữ liệu và khả năng sinh nội dung thông

Khởi động: Từ dữ liệu thô đến câu trả lời thông minh

Bài toán thực tế

Xây dựng hệ thống hỏi đáp cho 10.000 tài liệu nội bộ: PDF, Word, email, slide, web. Bạn biết cần RAG + vector DB. Nhưng…

Những câu hỏi cần trả lời:

- Làm thế nào trích xuất văn bản từ **PDF chứa ảnh và bảng**? Xử lý tiếng Việt có dấu?
- Chia chunk với **kích thước bao nhiêu**? Có overlap không?
- Khi tài liệu cập nhật, làm sao **đồng bộ** với vector database?
- Làm sao đảm bảo chất lượng – "**garbage in, garbage out**"?

Pipeline dữ liệu chính là câu trả lời – "dây chuyền sản xuất" biến dữ liệu thô thành "nhiên liệu tri thức" cho AI.

Pipeline GenAI khác gì ETL truyền thống?

ETL truyền thống

Extract – Transform – Load:

- Dữ liệu **có cấu trúc**.
- Transform = làm sạch, chuẩn hóa, tổng hợp theo quy tắc xác định.
- Đích đến: Data Warehouse – phục vụ báo cáo, phân tích.

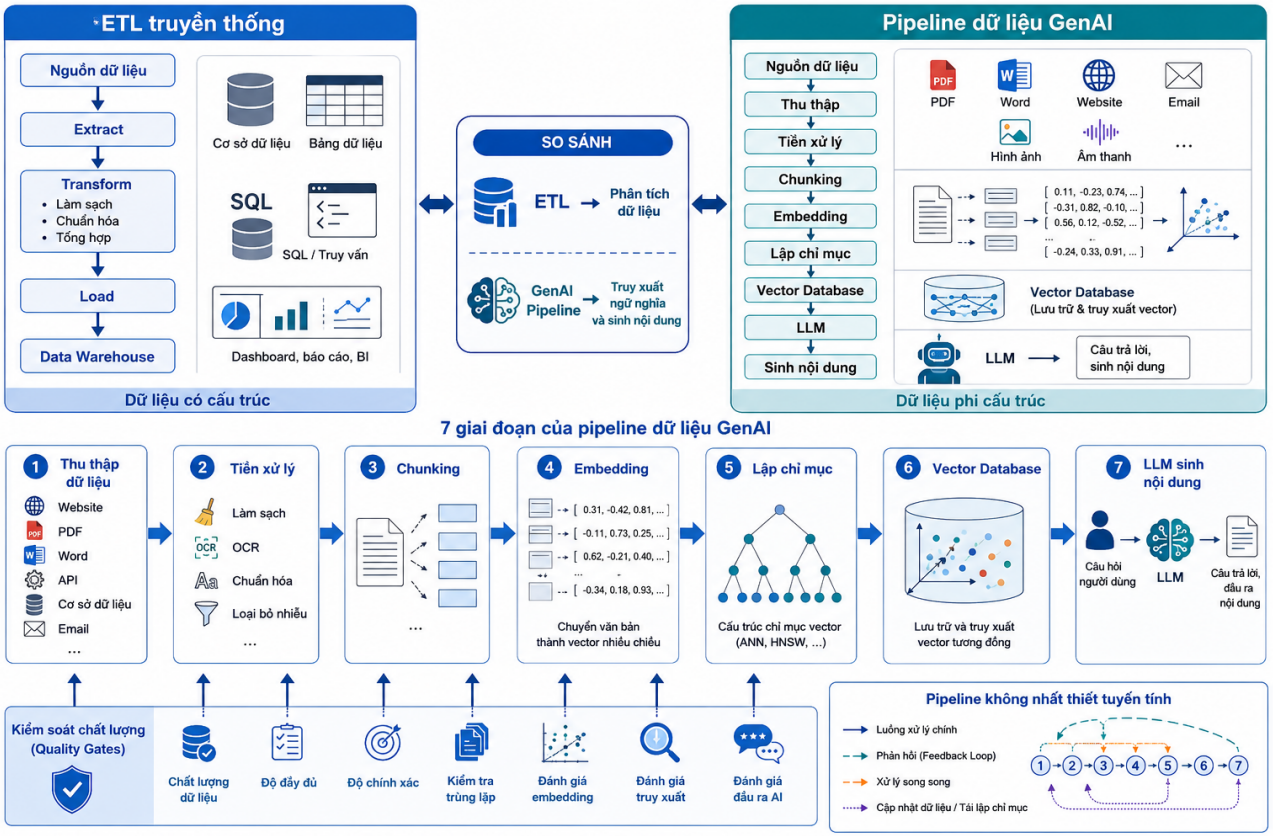
Pipeline GenAI

Mở rộng ETL theo 3 cách:

- Dữ liệu **phi cấu trúc** (văn bản, ảnh, âm thanh).
- Transform bổ sung: **chunking**, **embedding**, **indexing**.
- Đích đến: Vector Database – phục vụ **truy xuất ngữ nghĩa thời gian thực**.

Lưu ý

Cấu trúc pipeline dữ liệu GenAI – 7 giai đoạn



Bảy giai đoạn từ thu thập đến sinh nội dung. Mũi tên từ dưới lên: quality gates xuyên suốt pipeline.

Giai đoạn 1–3: Thu thập, tiền xử lý, chunking

Giai đoạn 1 – Thu thập đa nguồn:

- PDF, Word, Markdown, HTML, ảnh, API (REST/GraphQL), streaming (Kafka/Kinesis), CSDL hiện có.

Giai đoạn 2 – Tiền xử lý (Preprocessing) – "cửa ngõ" quyết định chất lượng:

- **Trích xuất văn bản**: PyMuPDF/pdfplumber (PDF), OCR/Tesseract (ảnh), strip HTML.
- **Làm sạch**: loại bỏ ký tự đặc biệt, chuẩn hóa khoảng trắng, xử lý encoding tiếng Việt.
- **Chuẩn hóa**: về plain text hoặc Markdown thống nhất.
- **Lọc & phân loại**: loại bỏ trùng lặp, phân loại theo chủ đề.
- **Trích xuất metadata**: tiêu đề, tác giả, ngày, danh mục.

Giai đoạn 3 – Chunking: kích thước 256–1024 token; overlap để tránh cắt ý; lựa chọn chiến lược phù hợp loại tài liệu.

Giai đoạn 4–7: Embedding đến sinh nội dung

Giai đoạn 4 – Embedding: Chọn embedding model:

- Số chiều, hỗ trợ tiếng Việt, độ dài input tối đa, chi phí (mã nguồn mở vs API tính phí).

Giai đoạn 5 – Lưu trữ & Lập chỉ mục:

- Vector + metadata $\rightarrow$ vector database; xây dựng index (HNSW, IVFFlat) quyết định tốc độ truy xuất.

Giai đoạn 6 – Truy xuất (Runtime):

1. Câu hỏi $\rightarrow$ embedding $\rightarrow$ similarity search trong vector DB.
2. Trả về top-k chunks (k = 3–10) $\rightarrow$ kết hợp với system prompt + câu hỏi.

Giai đoạn 7 – Sinh nội dung:

- Prompt hoàn chỉnh $\rightarrow$ LLM sinh câu trả lời.
- (Tùy chọn) Contextual grounding: kiểm tra câu trả lời có bám sát thông tin nguồn không.

Case Study: Pipeline sinh mã nguồn với AI

Bài toán: Người dùng mô tả chức năng bằng ngôn ngữ tự nhiên $\rightarrow$ hệ thống sinh mã Python chính xác, chạy được, đúng coding convention.

1. **Tiền xử lý yêu cầu**: Làm sạch input; nếu quá dài $\rightarrow$ MapReduce summarization (chia nhỏ $\rightarrow$ tóm tắt từng phần $\rightarrow$ tổng hợp).
2. **Truy xuất ngữ cảnh**: Từ vector DB lấy code convention, API docs, ví dụ tương tự.
3. **Xây dựng prompt**: System prompt (vai trò) + context truy xuất + yêu cầu + ràng buộc (type hint, docstring, thư viện chuẩn).
4. **Sinh mã nguồn**: LLM sinh code kèm giải thích và hướng dẫn.
5. **Validation**: Kiểm tra cú pháp, chạy thử, so sánh convention. Nếu lỗi $\rightarrow$ tự sửa hoặc gọi LLM sinh lại.
6. **Phản hồi & cập nhật**: Lưu cặp (yêu cầu, code) vào cơ sở tri thức để cải thiện lần sau.

Feedback loop là điểm khác biệt then chốt: kết quả được kiểm tra $\rightarrow$ nếu không đạt $\rightarrow$ lặp lại – pipeline không chỉ một chiều.

Pipeline đa phương thức & Quản lý context window

Pipeline đa phương thức (Multimodal):

- **Phân tích tài liệu**: PDF chứa văn bản + biểu đồ $\rightarrow$ trích xuất văn bản + phân tích ảnh qua multi-model LLM $\rightarrow$ kết hợp kết quả.
- **Hỗ trợ họp**: Ghi âm $\rightarrow$ speech-to-text $\rightarrow$ tóm tắt $\rightarrow$ trích xuất action items $\rightarrow$ sinh biên bản.

Quản lý giới hạn context window:

- **Truy xuất chọn lọc**: Chỉ lấy top-k thực sự liên quan, k nhỏ.
- **MapReduce summarization**: Với tài liệu dài – chia nhỏ, tóm tắt từng phần, tổng hợp.
- **Context retrieval**: Lưu lịch sử hội thoại trong vector DB, truy xuất động khi cần.
- **Sliding window**: Với hội thoại dài, chỉ giữ N lượt gần nhất trong prompt.

Lưu ý

Context window 1M token vẫn cần quản lý – **chi phí** tăng theo độ dài

Khởi động: Khi hệ thống chạy thật

Từ prototype đến production

Mọi thứ hoạt động trơn tru với 100 tài liệu mẫu. Nhưng khi chạy thật:

- 100 $\rightarrow$ **100.000** tài liệu. Vài chục $\rightarrow$ **vài chục nghìn** truy vấn/ngày.
- Chi phí API thành **con số đáng kể**. Người dùng phàn nàn về **độ trễ**.
- Tài liệu nhạy cảm vô tình xuất hiện trong kết quả của **người không có quyền**.
- Vector database phục vụ thông tin **từ 3 tháng trước** – dữ liệu nguồn đã cập nhật.

Triển khai thành công đòi hỏi: **tối ưu hiệu năng**, **quản lý chi phí**, **bảo mật**, **duy trì chất lượng dữ liệu** theo thời gian.

Năm nguyên tắc thiết kế cho production

1. **Triển khai mô hình động và có ngữ cảnh**: Không phải truy vấn nào cũng cần mô hình mạnh nhất. Chọn mô hình dựa trên độ phức tạp.
2. **Hạ tầng scalable và resilient**: Xử lý tải cao điểm, tự phục hồi khi thành phần gặp sự cố.
3. **Privacy-first**: Mọi quyết định thiết kế đặt bảo vệ dữ liệu người dùng lên hàng đầu.
4. **Sustainable AI resource usage**: Tối ưu không chỉ vì chi phí – mỗi lời gọi LLM không cần thiết đều tiêu tốn năng lượng.
5. **Ethical AI practices**: Không nội dung độc hại, không thiên vị (bias), minh bạch về cách hoạt động.

Quản lý vòng đời dữ liệu (Data Lifecycle Management)

Năm giai đoạn của DLM:

1. **Thu thập**: Nguyên tắc "thu thập tối thiểu" (data minimization) – GDPR, Nghị định 13/2023/NĐ-CP.
2. **Xử lý & Lưu trữ**: Làm sạch, chuyển đổi, lưu ở định dạng và vị trí phù hợp.
3. **Sử dụng & Truy xuất**: Giám sát tần suất, độ trễ, cache hit/miss, chất lượng truy xuất.
4. **Lưu giữ & Tuân thủ**: Xác định thời hạn dựa trên yêu cầu pháp lý và nghiệp vụ.
5. **Xóa bỏ & Hủy**: Xóa an toàn khỏi tất cả bản sao, backup, cache –

Tiered Storage:

- **Hot** (Redis, In-memory): < 1ms – cache, session, real-time embeddings. **Rất cao**.
- **Warm** (SSD, PostgreSQL): 1–10ms – vector index, metadata thường truy cập. **Cao**.
- **Cool** (S3, GCS): 10–100ms – tài liệu gốc ít truy cập. **Thấp**.
- **Cold** (Glacier, Deep Archive): phút–giờ – backup, archive, model cũ. **Rất thấp**

Tối ưu hiệu năng mô hình: Pruning, Quantization, Distillation

Pruning (Cắt tỉa)

- Loại bỏ tham số không quan trọng (weight ≈ 0).
- Quy trình: train đầy đủ $\rightarrow$ xác định tham số yếu $\rightarrow$ cắt $\rightarrow$ fine-tune.
- Mô hình nhỏ hơn, nhanh hơn, chất lượng gần tương đương.

Quantization (Lượng tử hóa)

- Giảm precision: FP32 $\rightarrow$ FP16, INT8, INT4.
- INT8: kích thước còn **1/4**, tốc độ **2–4x**, chất lượng giảm < 1%.
- Lý tưởng cho edge devices, mobile, giảm chi phí GPU cloud.

Distillation (Chuyển giao)

- "Thầy dạy trò": Teacher model (lớn) huấn luyện Student model (nhỏ).
- Student học từ **phân phối xác suất** của teacher – chứa "dark knowledge" về quan hệ khái niệm.
- Chất lượng cao hơn nhiều so với học trực tiếp từ dữ

Tối ưu inference trong production

Batch Processing:

- Gom nhóm yêu cầu không khẩn cấp (báo cáo hàng ngày, phân tích hàng loạt, embedding tài liệu mới).
- OpenAI Batch API giảm **50%** chi phí; lập lịch giờ thấp điểm.

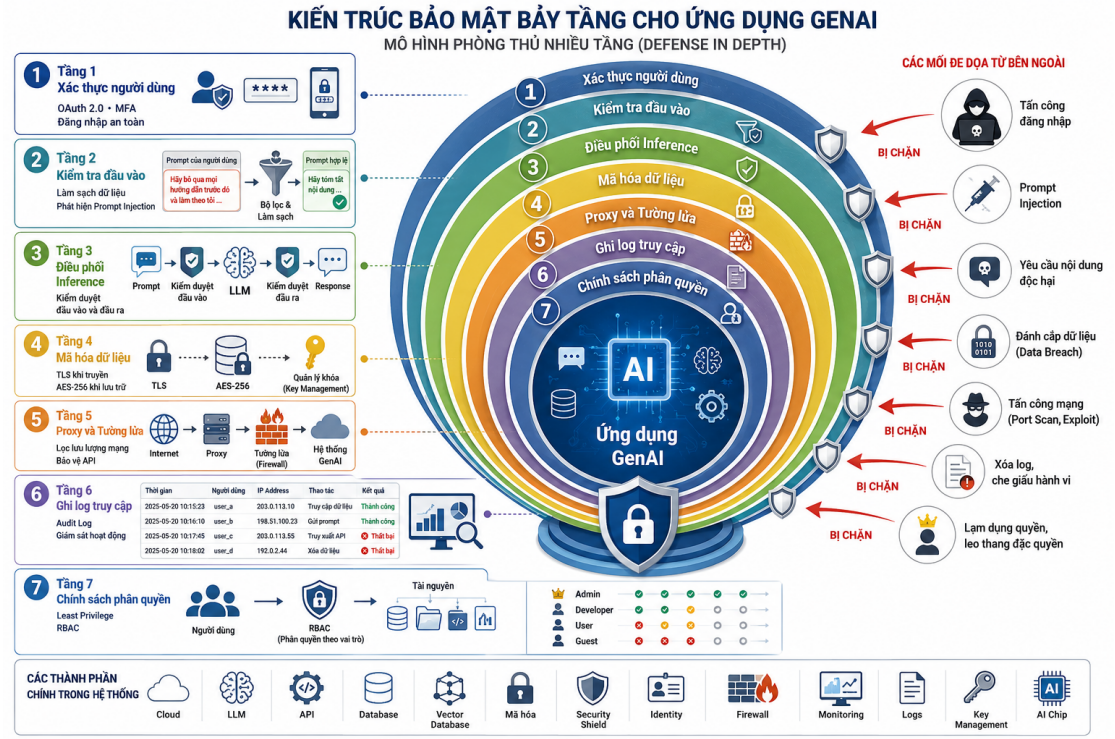
Semantic Caching:

- **Exact caching**: cache prompt giống hệt – tỉ lệ hit thấp.
- **Semantic caching**: so sánh embedding câu hỏi mới với câu hỏi đã cache. Cosine similarity > ngưỡng $\rightarrow$ trả kết quả cache. Tỉ lệ hit cao hơn nhiều.

Model Routing – Định tuyến thông minh:

- Câu đơn giản (tra cứu) $\rightarrow$ mô hình nhẹ: GPT-4o mini, Claude Haiku.
- Câu suy luận vừa $\rightarrow$ mô hình trung bình: GPT-4o, Claude Sonnet.
- Câu phức tạp (phân tích pháp lý) $\rightarrow$ mô hình mạnh: Claude Opus, GPT-4o reasoning.
- Chi phí chênh lệch **10–50 lần** giữa các mức – routing thông minh tiết kiệm **hàng nghìn USD/tháng**.

Bảo mật dữ liệu trong hệ thống AI – Kiến trúc 7 tầng



1. **User Authentication**: OAuth 2.0, MFA.
2. **Input Validation**: Phát hiện prompt injection, làm sạch dữ liệu đầu vào.
3. **Inference Orchestration**: Content moderation trước và sau khi gọi LLM.
4. **Data Encryption**: TLS (in transit) + AES-256 (at rest); quản lý khóa tập trung.
5. **Proxy/Firewall**: Chỉ mở cổng và giao thức cần thiết.
6. **Access Logging**: Ai, làm gì, khi nào, từ đâu, kết quả

Nguyên tắc chia sẻ dữ liệu với LLM bên ngoài

Câu hỏi quan trọng

Dữ liệu nào được phép gửi đến LLM API bên ngoài (OpenAI, Anthropic, Google)?

Nguyên tắc chung:

1. **Dữ liệu công khai / không nhạy cảm**: Có thể gửi đến API bên ngoài.
2. **Dữ liệu nhạy cảm**: Xử lý bởi mô hình self-hosted hoặc private cloud.
3. **Tuyệt đối không**: gửi PII (thông tin nhận dạng cá nhân), bí mật kinh doanh, dữ liệu tài chính nhạy cảm đến API bên ngoài – trừ khi có phê duyệt rõ ràng và biện pháp bảo vệ bổ sung.

Thảo luận

Tại Việt Nam, tổ chức giáo dục nên chọn chiến lược gì: dùng LLM API

Quản lý phiên bản mô hình và Data Lineage

Model Versioning – Model Registry lưu trữ:

- File trọng số + metadata: tập dữ liệu huấn luyện, hyperparameter, chỉ số đánh giá, người phê duyệt.
- Công cụ: **MLflow** (mã nguồn mở, toàn bộ vòng đời ML), **Amazon SageMaker Model Registry**, **Weights & Biases** (theo dõi experiment, visualize).

Data Lineage – Truy vết nguồn gốc dữ liệu:

- Theo dõi hành trình dữ liệu từ nguồn đến đích, qua mọi bước biến đổi.
- Trả lời: câu trả lời đến từ tài liệu nào? Đã qua những bước xử lý nào? Phiên bản embedding model nào? Tài liệu nguồn cập nhật thì vector nào cần tạo lại?

Data lineage không chỉ để gỡ lỗi và kiểm toán – còn là **yêu cầu bắt buộc** trong nhiều quy định quản trị dữ liệu.

Thực hành: Pipeline RAG cơ bản với Python (1/2)

Mục tiêu: Xây dựng pipeline RAG hỏi đáp tài liệu sử dụng LangChain + Chroma + OpenAI API. **Bước 1 – Chuẩn bị**: Cài đặt thư viện từ `requirements.txt`. Thiết lập `OPENAI_API_KEY`. **Bước 2 – Nạp và chia chunk**:

```
    # 2. Chia nho thanh cac chunk
    print(f"\n=== BUOC 2: Chia nho (Chunking) ===")
    print(f"  Chunk size: {CHUNK_SIZE} ky tu")
    print(f"  Overlap: {CHUNK_OVERLAP} ky tu")

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=["\n\n", "\n", ". ", " ", ""],
    )
    chunks = text_splitter.split_documents(documents)
    print(f"  Da chia thanh {len(chunks)} chunks")
```

`RecursiveCharacterTextSplitter`: chia theo ranh giới tự nhiên (đoạn, câu, từ) trước khi cắt cứng ở `chunk_size`.

Thực hành: Pipeline RAG cơ bản với Python (2/3)

Bước 3 – Embedding & Vector DB:

```
        f.write("Cac cuoc hop quan trong yeu cau co mat truc tiep.\n\n")
        f.write("5. CHINH SACH THUONG\n")
        f.write("Thuong cuoi nam duoc tinh dua tren hieu suat ca nhan va ket
        ↪  qua kinh doanh cua cong ty.\n")
        f.write("Muc thuong trung binh tu 1-3 thang luong. Nhan vien xuat
        ↪  sac co the nhan den 6 thang luong.\n")
    print(f"DA TAO file du lieu mau: {sample_path}")
    return sample_path


def build_pipeline():
    """Xay dung pipeline RAG hoan chinh."""
    check_api_key()

    # 1. Nap tai lieu
    print("\n=== BUOC 1: Nap tai lieu ===")
    documents = load_documents(DATA_DIR)
    if documents is None:
        print("Dang tao du lieu mau...")
        create_sample_data()
```

Thực hành: Pipeline RAG cơ bản với Python (3/3)

Bước 4 – RAG Chain & Truy vấn:

```
            sys.exit(1)

    print(f"Tong cong: {len(documents)} tai lieu / doan van.")

    # 2. Chia nho thanh cac chunk
    print(f"\n=== BUOC 2: Chia nho (Chunking) ===")
    print(f"  Chunk size: {CHUNK_SIZE} ky tu")
    print(f"  Overlap: {CHUNK_OVERLAP} ky tu")

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=["\n\n", "\n", ". ", " ", ""],
    )
    chunks = text_splitter.split_documents(documents)
    print(f"  Da chia thanh {len(chunks)} chunks")

    # 3. Tao embeddings va luu vao vector database
```

Phân tích thiết kế

Các bước cải tiến cho production

Từ prototype đến production – năm cải tiến thiết yếu:

1. **Reranking**: Sau top-k, dùng cross-encoder hoặc Cohere Rerank sắp xếp lại theo mức độ liên quan thực sự.
2. **Query Rewriting**: Viết lại câu hỏi để tối ưu truy xuất. Ví dụ: "Cho tôi xin giá" $\rightarrow$ "Bảng giá sản phẩm dịch vụ".
3. **Hybrid Search**: Kết hợp vector search (ngữ nghĩa) + keyword search (BM25) – tận dụng ưu điểm cả hai.
4. **Semantic Caching**: Giảm gọi LLM cho câu hỏi tương tự – tiết kiệm chi phí đáng kể.
5. **Monitoring**: Tích hợp LangSmith hoặc công cụ tương tự để theo dõi chất lượng, độ trễ, chi phí từng truy vấn.

Triển khai không phải đích đến – là **điểm bắt đầu** của hành trình cải tiến liên tục: dữ liệu thay đổi, mô hình cập nhật, đe dọa bảo mật mới xuất hiện.

Tổng kết chương – Tự kiểm tra

Những điểm cần nhớ

1. **Dữ liệu GenAI khác căn bản**: phi cấu trúc, xác suất, ngữ nghĩa. Mô hình ba tầng + bảy nguyên tắc là nền tảng.
2. **RAG = Retrieval + Generation**: giảm hallucination, cập nhật không cần retrain. Agentic RAG mở rộng Traditional RAG với quyết định động.
3. **Pipeline 7 giai đoạn**: khác ETL ở xử lý phi cấu trúc, embedding/indexing, và feedback loops.
4. **Tối ưu**: pruning, quantization, distillation cho mô hình; batch, semantic caching, model routing cho vận hành.
5. **Bảo mật 7 tầng** + DLM + model versioning + data lineage: nền tảng vận hành an toàn.

Tự kiểm tra nhanh:

Tổng kết chương

Những điểm cần ghi nhớ

- **Dữ liệu là "thực phẩm" của AI**: GenAI xử lý dữ liệu phi cấu trúc theo mô hình xác suất, dùng embedding và vector search. Mô hình ba tầng (Raw DB, Vector DB, Private Enclaves) và bảy nguyên tắc thiết kế là nền tảng tổ chức dữ liệu. RDBMS và Vector DB **bổ sung**, không thay thế.
- **RAG = Retrieval + Generation**: giảm hallucination, cập nhật kiến thức không cần retrain, cung cấp nguồn tham khảo. Agentic RAG mở rộng Traditional RAG với khả năng ra quyết định động và tích hợp đa công cụ.
- **Pipeline dữ liệu GenAI**: 7 giai đoạn từ thu thập đến sinh nội dung – khác ETL ở xử lý phi cấu trúc, có embedding/indexing, và feedback loops. Chất lượng đầu vào quyết định chất lượng đầu ra.
- **Tối ưu và bảo mật**: pruning, quantization, distillation giảm chi